

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 3
по дисциплине «Анализ алгоритмов»

Тема Графовые модели алгоритмов

Студент Атаев К.А.

Группа ИУ7-52Б

Преподаватели Волкова Л. Л., Строганов Д.В.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Рекурсия 5

1.2 Графы как математическая модель алгоритмов 5

1.3 Графовые модели алгоритмов 6

1.3.1 Граф управления 6

1.3.2 Информационный граф 6

1.3.3 Операционная история 6

1.3.4 Информационная история 7

2 Конструкторская часть **8**

2.1 Итеративный алгоритм 8

2.2 Рекурсивный алгоритм 9

3 Технологическая часть **10**

3.1 Средства реализации 10

3.2 Реализация алгоритмов 10

4 Исследовательская часть **11**

4.1 Граф управления 11

4.2 Информационный граф 13

4.3 Операционная история 15

4.4 Информационная история 16

4.5 Вывод 18

ЗАКЛЮЧЕНИЕ **19**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **20**

ПРИЛОЖЕНИЕ А **21**

ВВЕДЕНИЕ

В данной лабораторной работе рассматриваются графовые модели алгоритмов с целью выявления участков программного кода, которые могут быть выполнены параллельно. Анализ проводится на материале двух реализаций одной задачи — рекурсивного и нерекурсивного алгоритмов, полученных из предыдущей лабораторной работы.

Для достижения поставленной цели необходимо решить следующие задачи:

1) описать рекурсивную и нерекурсивную версии алгоритма согласно индивидуальному варианту;

2) представить обе реализации с использованием четырёх графовых моделей — графа управления, информационного графа, операционной истории и информационной истории;

3) определить участки программного кода, которые могут быть выполнены параллельно, либо обосновать отсутствие таких участков;

1 Аналитическая часть

1.1 Рекурсия

Согласно общепринятому определению рекурсии [1], рекурсией называют способ задания объекта, при котором он определяется через самого себя. В программировании рекурсивной является функция, вызывающая саму себя для решения подзадачи, аналогичной основной. Такой подход позволяет естественным образом описывать процессы, обладающие свойством самоподобия.

Формально рекурсивная функция может быть записана следующим образом (1.1):

$$F(x) = \begin{cases} F_0(x), & x \in B, \\ G\left(x, F(h(x))\right), & x \notin B, \end{cases} \tag{1.1}$$

где B — множество базовых случаев, при которых дальнейшее разбиение задачи не требуется, $h(x)$ — преобразование аргумента, ведущее к подзадаче, а G — правило получения результата для текущего уровня вычисления.

Корректная рекурсивная конструкция всегда включает:

— по крайней мере один базовый случай;

— механизм гарантированного приближения к базовому случаю;

1.2 Графы как математическая модель алгоритмов

Математически граф G определяется множеством вершин (1.2):

$$X = \{x_1, x_2, \ldots, x_n\}, \tag{1.2}$$

и множеством рёбер (1.3):

$$A = \{a_1, a_2, \ldots, a_m\}. \tag{1.3}$$

Таким образом, граф задаётся парой множеств (1.4):

$$G = (X, A). \tag{1.4}$$

Использование графов для анализа алгоритмов позволяет:

— описывать структуру вычислений;

— фиксировать зависимости по управлению;

— выявлять зависимости по данным;

— определять независимые компоненты алгоритма;

— находить возможные области для параллельного выполнения;

1.3 Графовые модели алгоритмов

В данной работе применяются четыре графовые модели: граф управления, информационный граф, операционная история и информационная история;

1.3.1 Граф управления

Граф управления (Control Flow Graph, CFG) — ориентированный граф (1.5):

$$G_{\text{ctrl}} = (V_{\text{ctrl}}, A_{\text{ctrl}}), \tag{1.5}$$

где:

— V_{ctrl} — множество операторов программы;

— A_{ctrl} — множество дуг, отражающих возможные переходы управления;

Дуги показывают последовательность выполнения операций, а также ветвления, циклы и рекурсивные возвраты. Если между двумя вершинами отсутствует путь, эти части алгоритма независимы по управлению.

1.3.2 Информационный граф

Информационный граф (Data Flow Graph, DFG) — ориентированный граф (1.6):

$$G_{\text{data}} = (V_{\text{ops}}, A_{\text{data}}), \tag{1.6}$$

где вершины соответствуют операциям, а дуга (u, v) существует, если результат операции u используется операцией v.

Такой граф показывает зависимости по данным. Отсутствие зависимости между двумя операциями означает возможность их параллельного выполнения.

1.3.3 Операционная история

Операционная история представляет собой частично упорядоченное множество (1.7):

$$H_{\text{ops}} = (O, \prec), \tag{1.7}$$

где O — множество операций, а $\prec$ — отношение строгого частичного порядка, определяющее их обязательный порядок выполнения.

Если для двух операций отношение порядка не задано, их можно выполнять независимо или параллельно.

1.3.4 Информационная история

Информационная история описывает изменения данных и задаётся структурой (1.8):

$$H_{\text{data}} = \{(d, t_{\text{create}}, U(d))\}, \tag{1.8}$$

где:

— d — значение данных или переменная;

— t_{create} — операция, породившая это значение;

— $U(d)$ — множество операций, использующих его;

Эта модель позволяет выявлять конфликты при обращении к данным, определять независимые цепочки преобразований и устанавливать возможность их параллельного выполнения.

Вывод

В данном разделе были рассмотрены понятия рекурсии и графов.

2 Конструкторская часть

В данном разделе приведены схемы алгоритмов, реализующих вывод всех целых чисел в диапазоне от A до B включительно при условии $A \leqslant B$.

2.1 Итеративный алгоритм

Схема алгоритма приведена на рисунке 2.1.

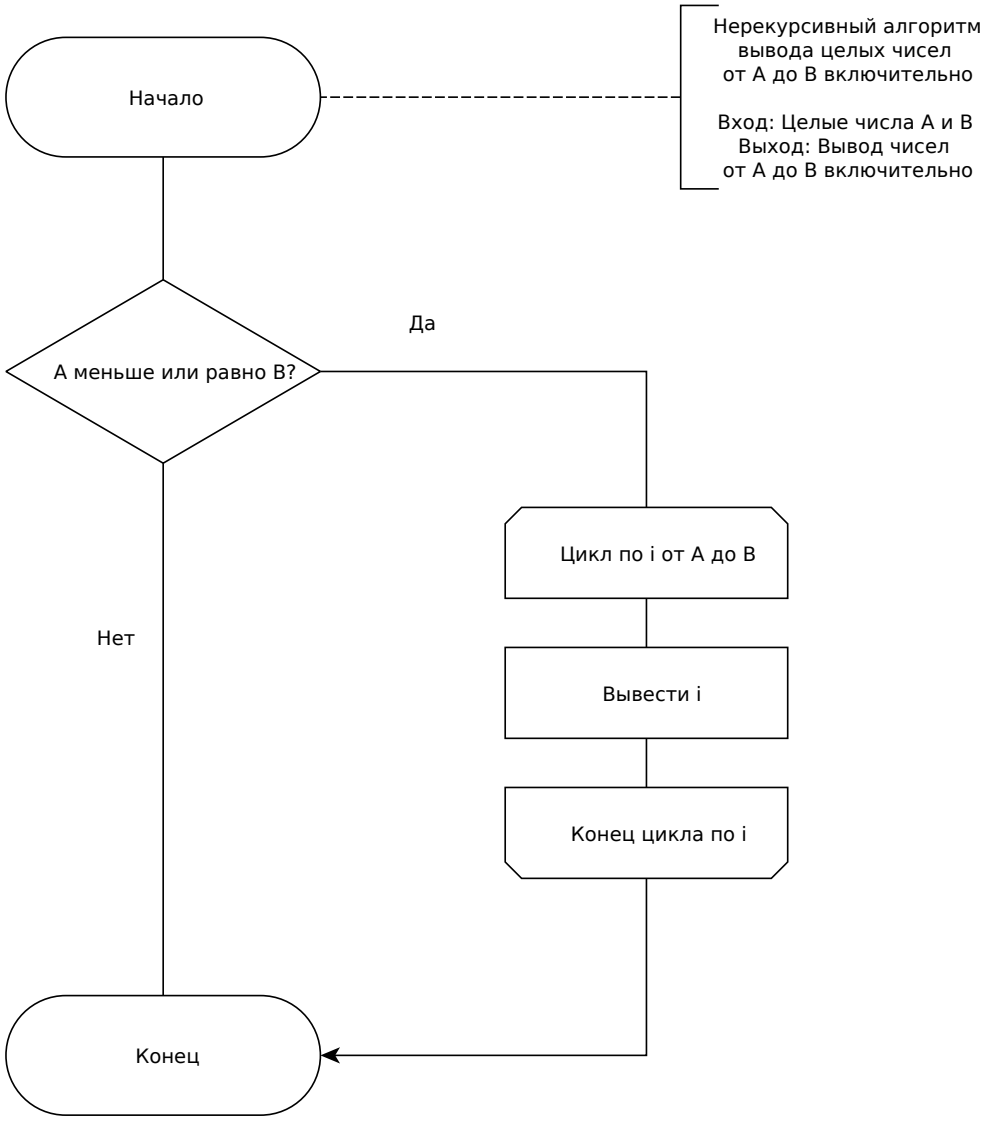


Рисунок 2.1 — Схема итеративного алгоритма

2.2 Рекурсивный алгоритм

Схема алгоритма приведена на рисунке 2.2.

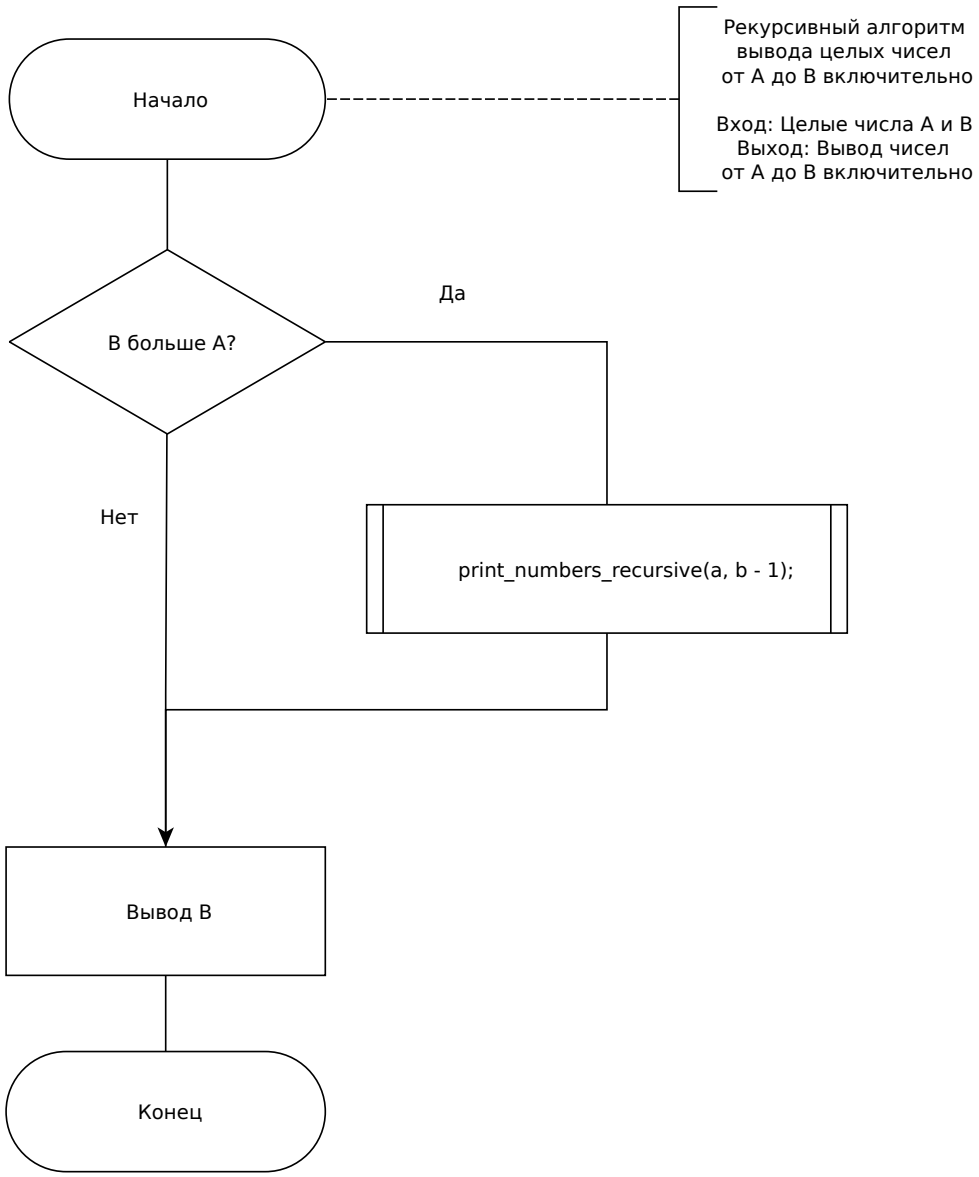


Рисунок 2.2 — Схема рекурсивного алгоритма

Вывод

В данном разделе были представлены схемы рекурсивного и итеративного алгоритмов

3 Технологическая часть

Здесь приведены средства реализации, реализации алгоритмов и функциональные тесты

3.1 Средства реализации

Для реализации был выбран язык C++, а в качестве среды разработки использовалась Visual Studio Code с компилятором g++ [2].

3.2 Реализация алгоритмов

В этом разделе представленыы реализации алгоритмов рекурсивного и нерекурсивного вывода чисел от A до B включительно.

В листингах А.1 —А.2 представлены реализации алгоритмов.

Вывод

В технологической части были представлены листинги двух алгоритмов, реализующих вывод всех целых чисел в диапазоне от A до B включительно.

4 Исследовательская часть

В данном разделе приведены графовые модели алгоритмов реализующих вывод всех целых чисел в диапазоне от A до B включительно при условии $A \leqslant B$. В рекурсивном алгоритме для рекурсивного вызова используется вершина 5, а 5б для присваивание его результата переменной.

Вершины графа представляют собой строчки кода представленного в листингах А.1 — А.2

4.1 Граф управления

Граф управления рекурсивного алгоритма приведен на рисунке 4.1.

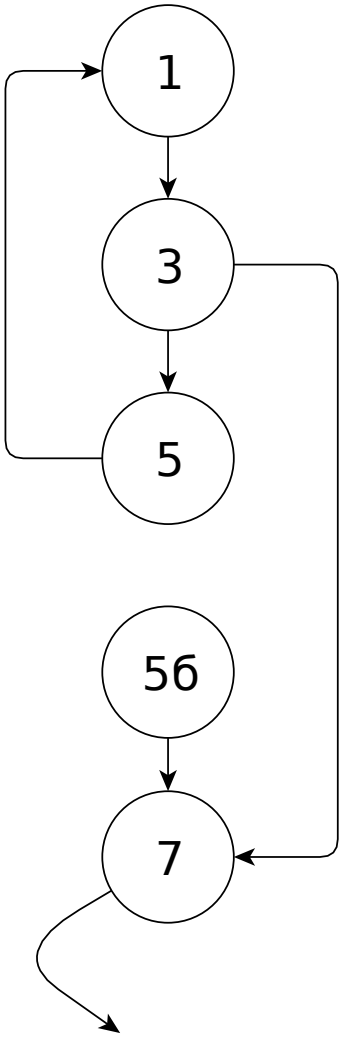


Рисунок 4.1 — Граф управления рекурсивного алгоритма

На рисунке 4.2 приведен граф управления итеративного алгоритма.

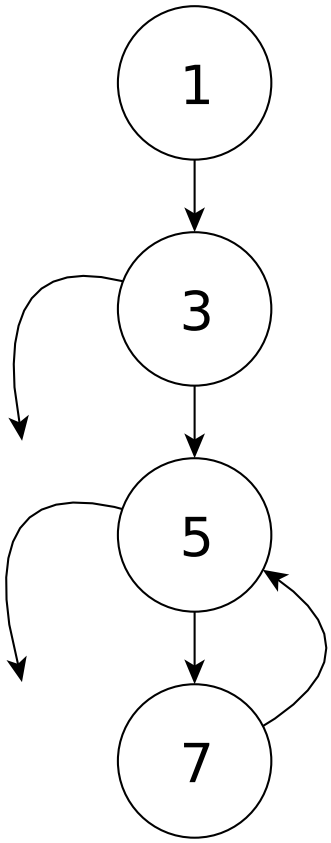


Рисунок 4.2 — Граф управления итеративного алгоритма

4.2 Информационный граф

На рисунке 4.3 приведен информационный граф рекурсивного алгоритма

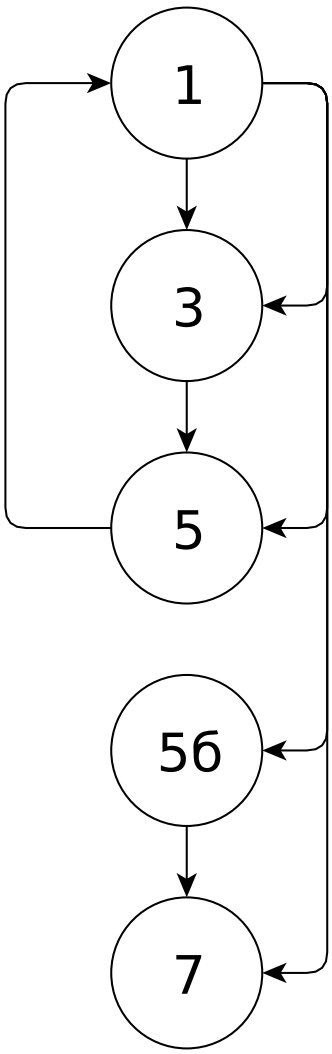


Рисунок 4.3 — Информационный граф рекурсивного алгоритма

На рисунке 4.4 приведен информационный граф итеративного алгоритма

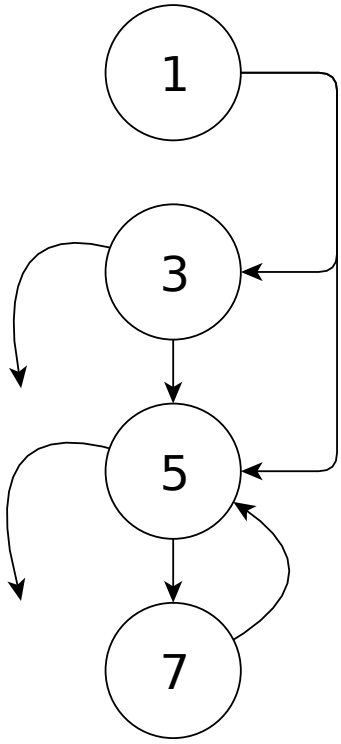


Рисунок 4.4 — Информационный граф итеративного алгоритма

4.3 Операционная история

Для построения операционной истории рекурсивного алгоритма в качестве входных данных использовались значения $A = 1$ и $B = 7$

На рисунке 4.5 приведена операционная история рекурсивного алгоритма

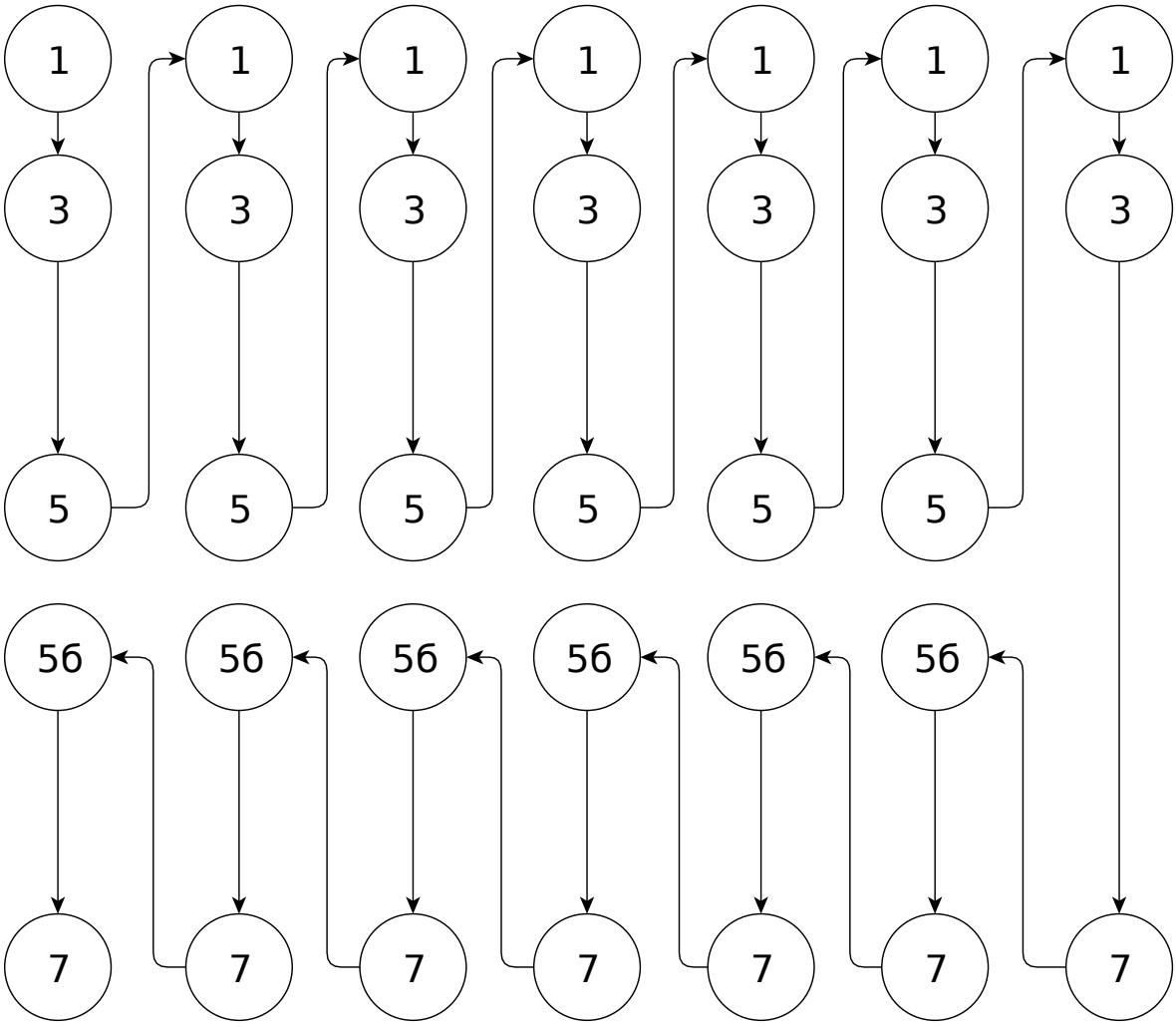


Рисунок 4.5 — Операционная история рекурсивного алгоритма

Для построения операционной истории итеративного алгоритма в качестве входных данных использовались значения $A = 1$ и $B = 6$

На рисунке 4.6 приведена операционная история итеративного алгоритма

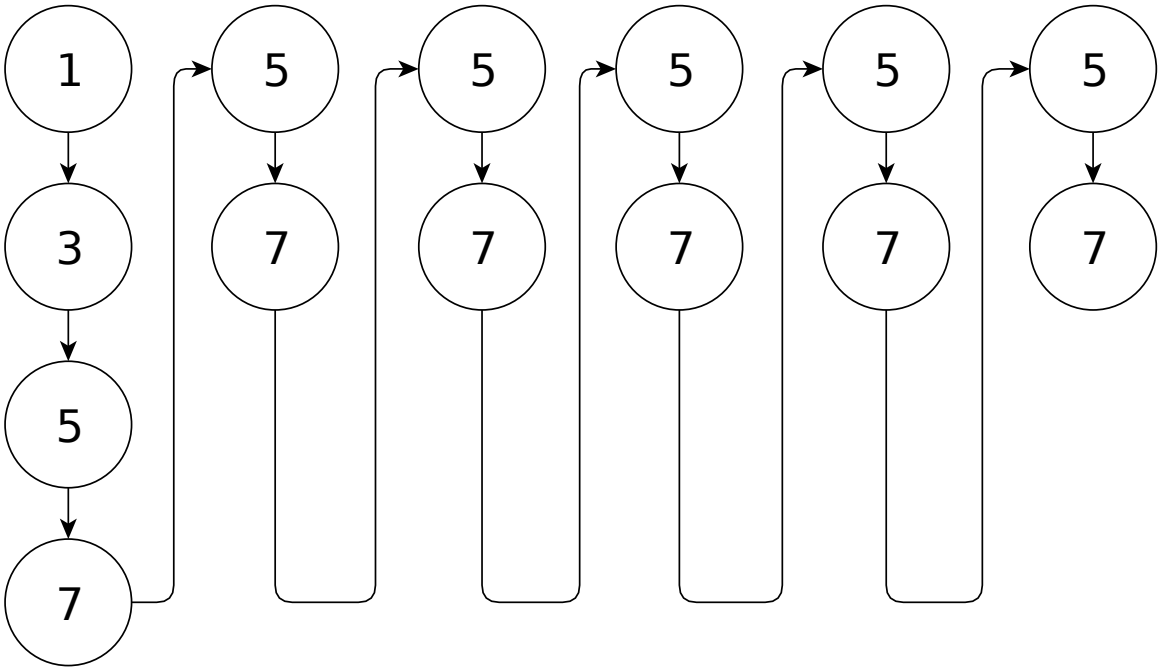


Рисунок 4.6 — Операционная история итеративного алгоритма

4.4 Информационная история

Для построения информационной истории рекурсивного алгоритма в качестве входных данных использовались значения $A = 1$ и $B = 7$

На рисунке 4.7 приведена информационная история рекурсивного алгоритма

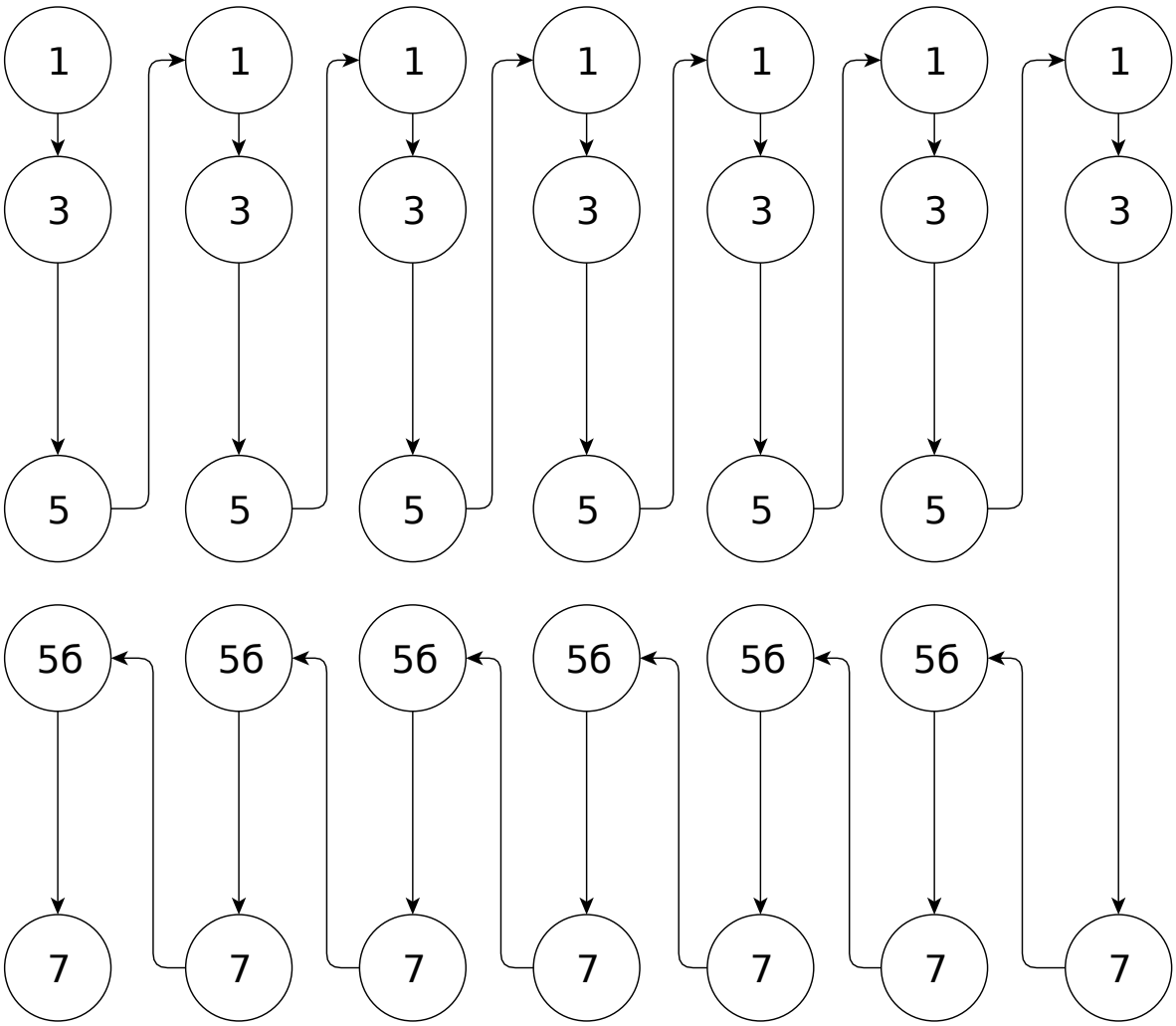


Рисунок 4.7 — Информационная история рекурсивного алгоритма

Для построения информационной истории итеративного алгоритма в качестве входных данных использовались значения $A = 1$ и $B = 6$

На рисунке 4.8 приведена информационная история рекурсивного алгоритма

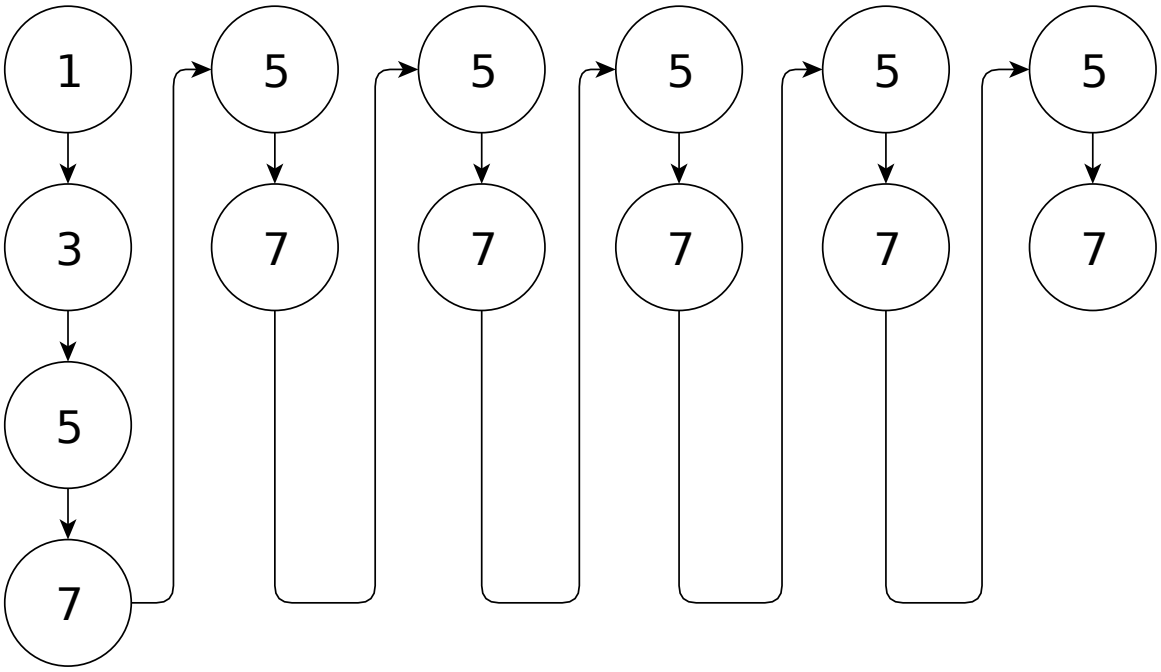


Рисунок 4.8 — Информационная история итеративного алгоритма

4.5 Вывод

Проведённый анализ графовых моделей рекурсивного и итеративного алгоритмов показал отсутствие шагов, допускающих параллельное выполнение. Все вершины графов соединены между собой через информационные или операционные зависимости.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был проведён анализ графовых моделей алгоритмов с целью выявления участков программного кода, допускающих параллельное выполнение.

Все цели лабораторной работы были достигнуты, а задачи выполнены.

1) описаны рекурсивная и нерекурсивная версии алгоритма согласно индивидуальному варианту;

2) представлены обе реализации с использованием четырёх графовых моделей — графа управления, информационного графа, операционной истории и информационной истории;

3) обосновано отсутствие участков программного кода, которые могут быть выполнены параллельно, на основе анализа зависимостей и структуры алгоритмов;

Проведённый анализ графовых моделей как рекурсивной, так и итеративной реализаций алгоритма не выявил участков, допускающих параллельное выполнение. Все вершины графов обладают информационными или операционными зависимостями, что определяет строго последовательный порядок выполнения операций в программе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Свейгарт Эл. Рекурсивная книга о рекурсии. — СПб.: Питер, 2023. — С. 26—197.
2. Документация Microsoft C++ [Электронный ресурс]. — Режим доступа: https://learn.microsoft.com/ru-ru/cpp/cpp/?view=msvc-170 (дата обращения: 29.10.2025).

ПРИЛОЖЕНИЕ А

```
void print_numbers_recursive(int a, int b)
{
  if (b > a)
  {
    print_numbers_recursive(a, b - 1);
  }
  std::cout << b << " ";
}
```

Листинг А.1 — Рекурсивная функция вывода чисел в диапазоне от A до B

```
void print_numbers_iterative(int a, int b)
{
  if (a <= b)
  {
    for (int i = a; i <= b; i++)
    {
      std::cout << i << " ";
    }
  }
}
```

Листинг А.2 — Итеративная функция вывода чисел в диапазоне от A до B